Dict
A dict is a way of representing a **constraint** as a **value** instead of a **context**

```
toString :: Show a => a -> String                    Constraint as context Show a => ...
toString v = show v

toString :: Dict (Show a) -> a -> String             Constraint as value Dict (Show a)
-> ...
toString Dict v = show v
```

usage: recover properties of an **existential (aka opaque) type**
An **existential type** does not have a type **signature** of the **inner type**, so a dict is needed to recover the constraints
e.g.

```
data SomeValue = forall a. SomeValue (Value a)          opaque container
class DoSomething x where doSomething :: x -> x         some type class

f :: SomeValue -> SomeValue
f (Some1 v) = SomeValue $ doSomething v                 BAD. How to get
(DoSomething (Value a))?

valueProperties :: Value a -> Dict (DoSomething (Value a))
f :: SomeValue -> SomeValue
f (Some1 v) = case valueProperties v of Dict ->
  SomValue $ doSomething v                              OK. (DoSomething (Value
a)) recovered using Dict
```

Define Dict

```
{-# LANGUAGE GADTs #-}
{-# LANGUAGE PolyKinds #-}
{-# LANGUAGE RankNTypes #-}

import Data.Kind

data Dict (c :: Constraint) where
  Dict :: (c) => Dict c

withDict :: Dict c -> ((c) => a) -> a
withDict Dict x = x
```

e.g. Dict

```
{-# LANGUAGE FlexibleInstances #-}

data Value a where
  ValueStr :: String -> Value String
  ValueInt :: Int -> Value Int

-- Opaque container
data SomeValue = forall a. SomeValue (Value a)
```

```
class DoSomething x where
  doSomething :: x -> x

instance DoSomething (Value String) where doSomething (ValueStr x)
= ValueStr $ x <> x
instance DoSomething (Value Int) where doSomething (ValueInt x) =
ValueInt $ 2 * x

valueProperties :: Value a -> Dict (DoSomething (Value a))
valueProperties (ValueStr _) = Dict
valueProperties (ValueInt _) = Dict

doSomethingWithSomeValue :: SomeValue -> SomeValue
doSomethingWithSomeValue (SomeValue v) =
             -- OK
  withDict (valueProperties v) $ SomeValue $ doSomething v
doSomethingWithSomeValue (SomeValue v) =
             -- OK with case
  case (valueProperties v) of Dict -> SomeValue $ doSomething v
-- doSomethingWithSomeValue (SomeValue v) =
             -- ERROR "No instance for (DoSomething x) arising from
a use of ‘doSomething’"
  SomeValue $ doSomething v
```

==========

constraints

Constraints: https://hackage.haskell.org/package/constraints/docs/Data-Constraint.html

Data.Constraint

```
data Dict :: Constraint -> * where
  Dict :: a => Dict a
class HasDict c e | e -> c where
  evidence :: e -> Dict c
withDict :: HasDict c e => e -> (c => r) -> r
(\\) :: HasDict c e => (c => r) -> e -> r

-- Sub
newtype a :- b = Sub (a => Dict b)
type (⊢) = (:-)
class (p => q) => p |- q
class (p, q) => p & q
weaken1 :: (a, b) :- a
weaken2 :: (a, b) :- b
contract :: a :- (a, a)
strengthen1 :: Dict b -> (a :- c) -> a :- (b, c)
strengthen2 :: Dict b -> (a :- c) -> a :- (c, b)
```

```
(&&&) :: (a :- b) -> (a :- c) -> a :- (b, c)
(***) :: (a :- b) -> (c :- d) -> (a, c) :- (b, d)
trans :: (b :- c) -> (a :- b) -> a :- c
refl :: a :- a
implied :: forall a b. (a => b) => a :- b

class Any => Bottom where
  no :: a
top :: a :- ()
bottom :: Bottom :- a
mapDict :: (a :- b) -> Dict a -> Dict b
unmapDict :: (Dict a -> Dict b) -> a :- b

-- Reflection
class Class b h | h -> b where
  cls :: h :- b
class b :=> h | h -> b where
  ins :: b :- h
```

==========

```
This
is
code
Monaco 14
```